

ООП (конструкторы)

Введение в конструкторы

Если тип класса является совокупностью, мы можем использовать инициализацию совокупности для непосредственной инициализации типа класса:

```
struct Foo // Foo is an aggregate
{
    int x {};
    int y {};
};

int main()
{
    Foo foo { 6, 7 }; // uses aggregate initialization

    return 0;
}
```

При инициализации агрегата выполняется инициализация каждого элемента (элементы инициализируются в том порядке, в котором они определены). Таким образом, когда в приведенном выше примере создается экземпляр `foo`, `foo.x` инициализируется значением 6, а `foo.y` инициализируется значением 7.

Однако, как только мы делаем какие-либо переменные-элементы закрытыми (чтобы скрыть наши данные), наш тип класса перестаёт быть совокупностью (потому что совокупности не могут иметь закрытые элементы). Это означает, что мы больше не можем использовать инициализацию совокупностей:

```
class Foo // Foo is not an aggregate (has private members)
{
    int m_x {};
    int m_y {};
};

int main()
{
    Foo foo { 6, 7 }; // compile error: can not use aggregate initialization

    return 0;
}
```

Запрет на инициализацию типов классов с закрытыми элементами через инициализацию совокупности имеет смысл по ряду причин:

- Инициализация совокупности требует знания о реализации класса (поскольку нужно знать, что представляют собой элементы и в каком порядке они были определены), чего мы намеренно стараемся избегать, скрывая элементы данных.
- Если бы у нашего класса был какой-то инвариант, мы бы полагались на то, что пользователь инициализирует класс таким образом, чтобы инвариант сохранялся.

Итак, как же инициализировать класс с закрытыми переменными-элементами? Сообщение об ошибке, выданное компилятором для предыдущего примера, даёт подсказку: «ошибка: нет подходящего конструктора для инициализации 'Foo'»

```
001_intro_02.cpp:9:20: error: no matching function for call to 'Foo::Foo(<brace-enclosed initializer list>)'
   9 |     Foo foo { 6, 7 }; // compile error: can not use aggregate initialization
     |           ^
001_intro_02.cpp:1:7: note: candidate: 'constexpr Foo::Foo()'
   1 | class Foo // Foo is not an aggregate (has private members)
```

Нам, должно быть, нужен соответствующий конструктор.

Конструкторы

Функция **Object()** { **собственный код** } — это специальная функция-элемент, которая автоматически вызывается после создания объекта неагрегированного типа.

При определении объекта неагрегированного типа класса компилятор проверяет, можно ли найти доступный конструктор, соответствующий значениям инициализации, предоставленным вызывающей стороной (если они есть).

- Если найден доступный подходящий конструктор, выделяется память для объекта, а затем вызывается функция-конструктор.
- Если не удаётся найти доступный соответствующий конструктор, возникает ошибка компиляции.

Ключевое понимание

Многие начинающие программисты не понимают, создают ли конструкторы объекты или нет. Они не создают — компилятор выделяет память для объекта до вызова конструктора. Затем конструктор вызывается для неинициализированного объекта.

Однако, если для набора инициализаторов не удаётся найти подходящий конструктор, компилятор выдаст ошибку. Таким образом, хотя конструкторы не создают объекты, отсутствие подходящего конструктора не позволит создать объект.

Помимо определения того, как может быть создан объект, конструкторы обычно выполняют две функции:

- Обычно они выполняют инициализацию любых переменных-элементов (через список инициализации элемента).
- Они могут выполнять другие функции настройки (с помощью операторов в теле конструктора). Это может включать в себя проверку значений инициализации на наличие ошибок, открытие файла или базы данных и т. д...

После завершения выполнения конструктора мы говорим, что объект «создан», и теперь он должен находиться в согласованном, пригодном для использования состоянии.

Обратите внимание, что агрегатам не разрешено иметь конструкторы, поэтому, если вы добавите конструктор в агрегат, он перестанет быть агрегатом.

Особенности именования конструкторов

В отличие от обычных функций-элементов, конструкторы должны называться по определённым правилам:

Конструкторы должны иметь то же имя, что и класс (с той же заглавной буквы). Для шаблонных классов это имя не включает параметры шаблона.

Конструкторы не имеют возвращаемого типа (даже void).

Поскольку конструкторы обычно являются частью интерфейса вашего класса, они, как правило, являются общедоступными.

Пример базового конструктора

Давайте добавим базовый конструктор к нашему примеру выше:

```
#include <iostream>

class Foo
{
private:
    int m_x {};
    int m_y {};

public:
    Foo(int x, int y) // here's our constructor function that takes two initializers
```

```

{
    std::cout << "Foo(" << x << ", " << y << ") constructed\n";
}

void print() const
{
    std::cout << "Foo(" << m_x << ", " << m_y << ")\n";
}
};

int main()
{
    Foo foo{ 6, 7 }; // calls Foo(int, int) constructor
    foo.print();

    return 0;
}

```

Теперь эта программа скомпилирует и выдаст результат:

```

Foo(6, 7) создан
Foo(0, 0)

```

Когда компилятор видит определение `Foo foo{ 6, 7 }`, он ищет соответствующий `Foo` конструктор, который принимает два `int` аргумента. `Foo(int, int)` подходит, поэтому компилятор разрешает это определение.

Во время выполнения, когда создаётся экземпляр `foo`, выделяется память для `foo`, а затем вызывается конструктор `Foo(int, int)` с параметром `x`, инициализированным значением 6, и параметром `y`, инициализированным значением 7. Затем выполняется тело функции-конструктора и выводится `Foo(6, 7) constructed`.

Когда мы вызовем функцию-элемент `print()`, вы заметите, что элементы `m_x` и `m_y` имеют значение 0. Это связано с тем, что, хотя наша функция-конструктор `Foo(int, int)` была вызвана, она не инициализировала элементы. Мы покажем, как это сделать, в следующем уроке.

Неявное преобразование аргументов конструктором

Компилятор выполняет неявное преобразование аргументов при вызове функции (при необходимости), чтобы соответствовать определению функции, в котором параметры имеют другой тип:

```

void foo(int, int)
{
}

int main()
{
    foo('a', true); // will match foo(int, int)

    return 0;
}

```

То же самое относится и к конструкторам: конструктор `Foo(int, int)` будет соответствовать любому вызову, аргументы которого неявно преобразуются в `int`:

```

class Foo
{
public:
    Foo(int x, int y)
    {
    }
};

int main()

```

```
{  
    Foo foo{ 'a', true }; // will match Foo(int, int) constructor  
  
    return 0;  
}
```

Конструкторы не должны быть постоянными

Конструктор должен иметь возможность инициализировать создаваемый объект, поэтому конструктор не должен быть константным.

```
#include <iostream>  
  
class Something  
{  
private:  
    int m_x{};  
  
public:  
    Something() // constructors must be non-const  
    {  
        m_x = 5; // okay to modify members in non-const constructor  
    }  
  
    int getX() const { return m_x; } // const  
};  
  
int main()  
{  
    const Something s{}; // const object, implicitly invokes (non-const) constructor  
  
    std::cout << s.getX(); // prints 5  
  
    return 0;  
}
```

Обычно функция-элемент, не являющаяся константной, не может быть вызвана для константного объекта. Однако в стандарте C++ прямо указано (согласно `class.ctor.general#5`), что константность не применяется к объекту, находящемуся в процессе создания, и вступает в силу только после завершения работы конструктора.

Конструкторы против сеттеров

Конструкторы предназначены для инициализации всего объекта в момент создания экземпляра. Сеттеры предназначены для присвоения значения одному элементу существующего объекта.

Списки инициализаторов элементов конструктора

Инициализация участника с помощью списка инициализации участника

Чтобы конструктор инициализировал элементы, мы используем список инициализации элементов (часто называемый «списком инициализации элементов»). Не путайте его с одноимённым «списком инициализации», который используется для инициализации совокупностей списком значений.

Списки инициализации элементов лучше всего изучать на примере. В следующем примере наш конструктор `Foo(int, int)` был обновлен для использования списка инициализации элементов для инициализации `m_x` и `m_y`:

```
#include <iostream>

class Foo
{
private:
    int m_x {};
    int m_y {};

public:
    Foo(int x, int y)
        : m_x { x }, m_y { y } // here's our member initialization list
    {
        std::cout << "Foo(" << x << ", " << y << ") constructed\n";
    }

    void print() const
    {
        std::cout << "Foo(" << m_x << ", " << m_y << ")\n";
    }
};

int main()
{
    Foo foo{ 6, 7 };
    foo.print();

    return 0;
}
```

Список инициализаторов элементов определяется после параметров конструктора. Он начинается с двоеточия (:), а затем перечисляет каждый элемент, который нужно инициализировать, вместе со значением инициализации для этой переменной, разделёнными запятыми. Здесь необходимо использовать прямую форму инициализации (предпочтительно с помощью фигурных скобок, но также можно использовать скобки) — инициализация копированием (с помощью знака равенства) здесь не работает. Также обратите внимание, что список инициализаторов элементов не заканчивается точкой с запятой.

Эта программа выдает следующий результат:

```
Foo(6, 7) constructed
Foo(6, 7)
```

При создании экземпляра `foo` элементы в списке инициализации инициализируются заданными значениями инициализации. В данном случае список инициализации элементов инициализирует `m_x` значением `x` (которое равно 6), а `m_y` — значением `y` (которое равно 7). Затем выполняется тело конструктора.

При вызове функции-элемента `print()` вы можете видеть, что `m_x` по-прежнему имеет значение 6 и `m_y` по-прежнему имеет значение 7.

Форматирование списка инициализаторов участников

C++ предоставляет широкие возможности для форматирования списков инициализации элементов по вашему усмотрению, так как не имеет значения, где вы ставите двоеточие, запятые или пробелы.

Все перечисленные ниже стили допустимы (и вы, скорее всего, встретите все три на практике):

```
Foo(int x, int y) : m_x { x }, m_y { y }
{
}
```

```
Foo(int x, int y) :
    m_x { x },
    m_y { y }
{
}
```

```
Foo(int x, int y)
    : m_x { x }
    , m_y { y }
{
}
```

Мы рекомендуем использовать третий стиль, описанный выше:

Поставьте двоеточие в строке после имени конструктора, так как это чётко отделяет список инициализаторов элементов от прототипа функции.

Сделайте отступ в списке инициализаторов элементов, чтобы было удобнее просматривать названия функций.

Если список инициализации элементов короткий/простой, все инициализаторы могут располагаться в одной строке:

```
Foo(int x, int y)
    : m_x { x }, m_y { y }
{
}
```

В противном случае (или если вы предпочитаете) каждую пару элементов и инициализаторов можно разместить на отдельной строке (начиная с запятой для выравнивания):

```
Foo(int x, int y)
    : m_x { x }
    , m_y { y }
{
}
```

Порядок инициализации участника

Согласно стандарту C++, элементы в списке инициализации элементов всегда инициализируются в том порядке, в котором они определены внутри класса (а не в том порядке, в котором они определены в списке инициализации элементов).

В приведённом выше примере, поскольку `m_x` определяется перед `m_y` в определении класса, `m_x` будет инициализирован первым (даже если он не указан первым в списке инициализаторов элементов).

Поскольку мы интуитивно ожидаем, что переменные будут инициализироваться слева направо, это может привести к незаметным ошибкам. Рассмотрим следующий пример:

```
#include <algorithm> // for std::max
```

```
#include <iostream>

class Foo
{
private:
    int m_x{};
    int m_y{};

public:
    Foo(int x, int y)
        : m_y { std::max(x, y) }, m_x { m_y } // issue on this line
    {
    }

    void print() const
    {
        std::cout << "Foo(" << m_x << ", " << m_y << ")\n";
    }
};

int main()
{
    Foo foo { 6, 7 };
    foo.print();

    return 0;
}
```

В приведенном выше примере наше намерение состоит в том, чтобы вычислить большее из значений инициализации, переданных через `std::max(x, y)`, а затем использовать это значение для инициализации обоих `m_x` и `m_y`. Однако на авторском компьютере печатается следующий результат:

```
Foo(262, 7)
```

Что произошло? Несмотря на то, что `m_y` указан первым в списке инициализации элементов, поскольку `m_x` определяется первым в классе, `m_x` инициализируется первым. И `m_x` инициализируется значением `m_y`, которое ещё не было инициализировано. Наконец, `m_y` инициализируется наибольшим из значений инициализации.

Чтобы предотвратить подобные ошибки, элементы в списке инициализации элементов должны быть перечислены в том порядке, в котором они определены в классе. Некоторые компиляторы выдают предупреждение, если элементы инициализируются в неправильном порядке.

Наилучшая практика

Переменные-элементы в списке инициализаторов элементов должны быть перечислены в том порядке, в котором они определены в классе.

Также рекомендуется избегать инициализации элементов с использованием значений других элементов (если это возможно). Таким образом, даже если вы допустите ошибку в порядке инициализации, это не будет иметь значения, поскольку между значениями инициализации нет зависимостей.

Список инициализаторов участников по сравнению с инициализаторами участников по умолчанию

Участники могут быть инициализированы несколькими различными способами:

- Если элемент указан в списке инициализаторов элементов, используется это значение инициализации
- В противном случае, если у элемента есть инициализатор по умолчанию, используется это значение инициализации
- В противном случае элемент инициализируется по умолчанию.

Это означает, что если элемент имеет инициализатор по умолчанию и указан в списке инициализаторов элементов для конструктора, то значение из списка инициализаторов элементов имеет приоритет.

Вот пример, показывающий все три метода инициализации:

```
#include <iostream>

class Foo
{
private:
    int m_x {};    // default member initializer (will be ignored)
    int m_y { 2 }; // default member initializer (will be used)
    int m_z;       // no initializer

public:
    Foo(int x)
        : m_x { x } // member initializer list
    {
        std::cout << "Foo constructed\n";
    }

    void print() const
    {
        std::cout << "Foo(" << m_x << ", " << m_y << ", " << m_z << ")\n";
    }
};

int main()
{
    Foo foo { 6 };
    foo.print();

    return 0;
}
```

На компьютере автора этот вывод:

```
Foo constructed
Foo(6, 2, 277)
```

Вот что происходит. При создании foo в списке инициализаторов элементов появляется только m_x, поэтому m_x сначала инициализируется значением 6. m_y не находится в списке инициализаторов элементов, но у него есть инициализатор по умолчанию, поэтому он инициализируется значением 2. m_z не находится ни в списке инициализаторов элементов, ни в списке инициализаторов по умолчанию, поэтому он инициализируется по умолчанию (что для базовых типов означает, что он остаётся неинициализированным). Таким образом, когда мы выводим значение m_z, мы получаем неопределённое поведение.

Тела функций конструкторов

Тело функции-конструктора чаще всего остаётся пустым. Это связано с тем, что мы в первую очередь используем конструктор для инициализации, которая выполняется с помощью списка инициализации элементов. Если это всё, что нам нужно сделать, то в теле конструктора не нужны никакие операторы.

Однако, поскольку операторы в теле конструктора выполняются после выполнения списка инициализации элементов, мы можем добавить операторы для выполнения любых других необходимых задач по настройке. В приведённых выше примерах мы выводим что-то на консоль, чтобы показать, что конструктор выполнен, но мы могли бы делать и другие вещи, например открывать файл или базу данных, выделять память и т. д...

Начинающие программисты иногда используют тело конструктора для присвоения значений элементам:

```
#include <iostream>
```

```

class Foo
{
private:
    int m_x { 0 };
    int m_y { 1 };

public:
    Foo(int x, int y)
    {
        m_x = x; // incorrect: this is an assignment, not an initialization
        m_y = y; // incorrect: this is an assignment, not an initialization
    }

    void print() const
    {
        std::cout << "Foo(" << m_x << ", " << m_y << ")\n";
    }
};

int main()
{
    Foo foo { 6, 7 };
    foo.print();

    return 0;
}

```

Хотя в этом простом случае это приведёт к ожидаемому результату, в случае, когда требуется инициализировать элементы (например, для элементов данных, которые являются константными или ссылочными), присваивание не работает.

Ключевое понимание

Как только список инициализаторов элементов завершает выполнение, объект считается инициализированным. Как только тело функции завершает выполнение, объект считается созданным.

Наилучшая практика

Для инициализации элементов лучше использовать список инициализации элементов, а не присваивать значения в теле конструктора.

Обнаружение и обработка недопустимых аргументов конструкторов

Рассмотрим следующий класс дробей:

```

class Fraction
{
private:
    int m_numerator {};
    int m_denominator {};

public:
    Fraction(int numerator, int denominator):
        m_numerator { numerator }, m_denominator { denominator }
    {
    }
};

```

Поскольку дробь — это числитель, делённый на знаменатель, знаменатель дроби не может быть равен нулю (иначе мы получим деление на ноль, которое математически не определено). Другими словами, в этом классе есть инвариант, который `m_denominator` не может быть 0.

Итак, что мы делаем, когда пользователь пытается создать дробь с нулевым знаменателем (например, **Fraction f { 1, 0 };**)?

Внутри списка инициализации элементов наши возможности по обнаружению и обработке ошибок довольно ограничены. Мы можем использовать условный оператор для обнаружения ошибки, но что дальше?

```
class Fraction
{
private:
    int m_numerator {};
    int m_denominator {};

public:
    Fraction(int numerator, int denominator):
        m_numerator { numerator }, m_denominator { denominator != 0.0 ? denominator :
??? } // what do we do here?
    {
    }
};
```

Мы могли бы изменить знаменатель на допустимое значение, но тогда пользователь получит **Fraction** без запрошенных значений, и у нас не будет возможности сообщить ему, что мы сделали что-то неожиданное. Поэтому мы обычно не пытаемся выполнить какую-либо проверку в списке инициализации элементов — мы просто инициализируем элементы переданными значениями, а затем пытаемся справиться с ситуацией.

Внутри тела конструктора мы можем использовать операторы, поэтому у нас есть больше возможностей для обнаружения и обработки ошибок. Это хорошее место для **assert** или **static_assert** проверки семантической корректности переданных аргументов, но это не поможет справиться с ошибками во время выполнения в рабочей сборке.

Если конструктор не может создать семантически корректный объект, мы говорим, что он не справился с задачей.

Когда конструкторы терпят неудачу (прелюдия)

Поскольку конструкторы являются функциями, они подвержены тем же проблемам при анализе и обработке ошибок.

Возможные стратегии устранения ошибок для функций:

- Устраните ошибку внутри функции.
- Передайте ошибку обратно вызывающему объекту для устранения.
- Остановите программу.
- Создайте исключение.

В большинстве случаев у нас недостаточно информации, чтобы полностью решить такие проблемы в рамках конструктора. Поэтому устранение проблемы, как правило, невозможно.

С помощью функций-элементов и неспециальных функций-элементов мы можем передать ошибку вызывающей стороне, чтобы она с ней разобралась. Но у конструкторов нет возвращаемого значения, поэтому у нас нет хорошего способа сделать это. В некоторых случаях мы можем добавить функцию-элемент **isValid()** (или перегруженное преобразование в **bool**), которая возвращает, находится ли объект в допустимом состоянии. Например, функция **isValid()** для **Fraction** вернёт **true** при **m_denominator != 0.0**. Но это означает, что вызывающая сторона должна не забывать вызывать функцию при каждом создании новой дроби. А наличие семантически некорректных объектов, которые можно получить доступ к ним, скорее всего, приведёт к ошибкам. Так что, хотя это лучше, чем ничего, это не самый лучший вариант.

В некоторых типах программ мы можем просто остановить всю программу и позволить пользователю перезапустить её с нужными входными данными... но в большинстве случаев это неприемлемо. Так что, скорее всего, нет.

И остаётся только выбросить исключение. Исключения полностью прерывают процесс создания, а это значит, что пользователь никогда не получит доступ к семантически некорректному объекту. Поэтому в большинстве случаев в таких ситуациях лучше всего выбросить исключение.

Ключевое понимание

Выброс исключения — это, как правило, лучшее, что можно сделать, когда конструктор выдаёт ошибку (и не может восстановиться).

Пока что мы будем исходить из того, что создание объекта нашего класса приводит к созданию семантически корректного объекта.

Для продвинутых читателей

Если исключения невозможны или нежелательны (либо потому, что вы решили их не использовать, либо потому, что вы ещё не изучили их), есть ещё один разумный вариант. Вместо того, чтобы позволять пользователю напрямую создавать класс, предоставьте функцию, которая либо возвращает экземпляр класса, либо указывает на ошибку.

В следующем примере наша функция `createFraction()` возвращает `std::optional<Fraction>` объект, который необязательно содержит допустимую `Fraction`. Если он содержит, то мы можем использовать эту дробь. Если нет, то вызывающая сторона может обнаружить это и справиться с этим.

```
#include <iostream>
#include <optional>

class Fraction
{
private:
    int m_numerator { 0 };
    int m_denominator { 1 };

    // private constructor can't be called by public
    Fraction(int numerator, int denominator):
        m_numerator { numerator }, m_denominator { denominator }
    {
    }

public:
    // Allow this function to access private members
    friend std::optional<Fraction> createFraction(int numerator, int denominator);
};

std::optional<Fraction> createFraction(int numerator, int denominator)
{
    if (denominator == 0)
        return {};

    return Fraction{numerator, denominator};
}

int main()
{
    auto f1 { createFraction(0, 1) };
}
```

```
if (f1)
{
    std::cout << "Fraction created\n";
}

auto f2 { createFraction(0, 0) };
if (!f2)
{
    std::cout << "Bad fraction\n";
}
}
```

Результатом выполнения данной программы будет:

```
Fraction created
Bad fraction
```

Конструкторы по умолчанию и аргументы по умолчанию

Конструктор по умолчанию — это конструктор, который не принимает аргументов. Как правило, это конструктор, который был определён без параметров.

Вот пример класса, который имеет конструктор по умолчанию:

```
#include <iostream>

class Foo
{
public:
    Foo() // default constructor
    {
        std::cout << "Foo default constructed\n";
    }
};

int main()
{
    Foo foo{}; // No initialization values, calls Foo's default constructor

    return 0;
}
```

При запуске приведённой выше программы создаётся объект типа Foo. Поскольку значения инициализации не были заданы, вызывается конструктор по умолчанию Foo(), который выводит:

```
Foo default constructed
```

Инициализация значений по сравнению с инициализацией по умолчанию для типов классов

Если у типа класса есть конструктор по умолчанию, то и инициализация значением, и инициализация по умолчанию вызовут конструктор по умолчанию. Таким образом, для такого класса, как класс Foo в приведённом выше примере, следующее по сути эквивалентно:

```
Foo foo{}; // value initialization, calls Foo() default constructor
Foo foo2;  // default initialization, calls Foo() default constructor
```

Однако, как мы уже рассмотрели на предыдущем уроке—инициализация значений безопаснее для совокупностей. Поскольку сложно определить, является ли тип класса совокупностью или нет, безопаснее использовать инициализацию значений для всего и не беспокоиться об этом.

Наилучшая практика

Предпочитайте инициализацию значения инициализации по умолчанию для всех типов классов.

Конструкторы с аргументами по умолчанию

Как и в случае со всеми функциями, крайние правые параметры конструкторов могут иметь аргументы по умолчанию.

Например:

```
#include <iostream>

class Foo
{
private:
    int m_x { };
    int m_y { };
}
```

```

public:
    Foo(int x=0, int y=0) // has default arguments
        : m_x { x }
        , m_y { y }
    {
        std::cout << "Foo(" << m_x << ", " << m_y << ") constructed\n";
    }
};

int main()
{
    Foo foo1{}; // calls Foo(int, int) constructor using default arguments
    Foo foo2{6, 7}; // calls Foo(int, int) constructor

    return 0;
}

```

Это печатает:

```

Foo(0, 0) constructed
Foo(6, 7) constructed

```

Если все параметры в конструкторе имеют аргументы по умолчанию, то конструктор является конструктором по умолчанию (потому что его можно вызвать без аргументов).

Перегруженные конструкторы

Поскольку конструкторы — это функции, их можно перегружать. То есть у нас может быть несколько конструкторов, чтобы мы могли создавать объекты разными способами:

```

#include <iostream>

class Foo
{
private:
    int m_x {};
    int m_y {};

public:
    Foo() // default constructor
    {
        std::cout << "Foo constructed\n";
    }

    Foo(int x, int y) // non-default constructor
        : m_x { x }, m_y { y }
    {
        std::cout << "Foo(" << m_x << ", " << m_y << ") constructed\n";
    }
};

int main()
{
    Foo foo1{}; // Calls Foo() constructor
    Foo foo2{6, 7}; // Calls Foo(int, int) constructor

    return 0;
}

```

Данный пример выведет:

```

Foo constructed
Foo(6, 7) constructed

```

Из вышесказанного следует, что у класса должен быть только один конструктор по умолчанию. Если предусмотрено несколько конструкторов по умолчанию, компилятор не сможет определить, какой из них следует использовать:

```
#include <iostream>

class Foo
{
private:
    int m_x {};
    int m_y {};

public:
    Foo() // default constructor
    {
        std::cout << "Foo constructed\n";
    }

    Foo(int x=1, int y=2) // default constructor
        : m_x { x }, m_y { y }
    {
        std::cout << "Foo(" << m_x << ", " << m_y << ") constructed\n";
    }
};

int main()
{
    Foo foo{}; // compile error: ambiguous constructor function call

    return 0;
}
```

В приведённом выше примере мы создаём экземпляр `foo` без аргументов, поэтому компилятор будет искать конструктор по умолчанию. Он найдёт два конструктора и не сможет определить, какой из них следует использовать. Это приведёт к ошибке компиляции.

Неявный конструктор по умолчанию

Если объект неагрегированного типа класса не имеет объявленных пользователем конструкторов, компилятор сгенерирует общедоступный конструктор по умолчанию (чтобы класс можно было инициализировать значением или по умолчанию). Этот конструктор называется неявным конструктором по умолчанию.

```
#include <iostream>

class Foo
{
private:
    int m_x{};
    int m_y{};

    // Note: no constructors declared
};

int main()
{
    Foo foo{};

    return 0;
}
```

У этого класса нет объявленных пользователем конструкторов, поэтому компилятор сгенерирует для нас неявный конструктор по умолчанию. Этот конструктор будет использоваться для создания экземпляра `foo`.

Неявный конструктор по умолчанию эквивалентен конструктору без параметров, без списка инициализации элементов и без операторов в теле конструктора. Другими словами, для приведённого выше Foo класса компилятор генерирует следующее:

```
public:
    Foo() // implicitly generated default constructor
    {
    }
```

Неявный конструктор по умолчанию полезен в основном, когда у нас есть классы, в которых нет элементов данных. Если в классе есть элементы данных, мы, вероятно, захотим сделать их инициализируемыми значениями, предоставленными пользователем, а неявного конструктора по умолчанию для этого недостаточно.

Использование = default для создания явно заданного конструктора по умолчанию

В случаях, когда мы пишем конструктор по умолчанию, эквивалентный неявно сгенерированному конструктору по умолчанию, мы можем вместо этого указать компилятору сгенерировать для нас конструктор по умолчанию. Такой конструктор называется конструктором по умолчанию с явным значением по умолчанию, и его можно сгенерировать с помощью синтаксиса = default:

```
#include <iostream>

class Foo
{
private:
    int m_x {};
    int m_y {};

public:
    Foo() = default; // generates an explicitly defaulted default constructor

    Foo(int x, int y)
        : m_x { x }, m_y { y }
    {
        std::cout << "Foo(" << m_x << ", " << m_y << ") constructed\n";
    }
};

int main()
{
    Foo foo{}; // calls Foo() default constructor

    return 0;
}
```

В приведённом выше примере, поскольку у нас есть объявленный пользователем конструктор (Foo(int, int)), неявный конструктор по умолчанию обычно не создаётся. Однако, поскольку мы указали компилятору создать такой конструктор, он будет создан. Этот конструктор впоследствии будет использоваться при создании экземпляра foo{}.

Наилучшая практика

Предпочитайте явно заданный конструктор по умолчанию (= default) конструктору по умолчанию с пустым телом.

Явно установленный по умолчанию конструктор по умолчанию против пустого пользовательского конструктора

Есть как минимум два случая, когда явно заданный по умолчанию конструктор ведёт себя иначе, чем пустой пользовательский конструктор.

1. При инициализации значения класса, если у класса есть пользовательский конструктор по умолчанию, объект будет инициализирован по умолчанию. Однако, если у класса есть конструктор по умолчанию, который не предоставлен пользователем (то есть конструктор по умолчанию, который либо неявно определён, либо определён с помощью `= default`), объект будет инициализирован нулями, прежде чем будет инициализирован по умолчанию.

```
#include <iostream>

class User
{
private:
    int m_a; // note: no default initialization value
    int m_b {};

public:
    User() {} // user-defined empty constructor

    int a() const { return m_a; }
    int b() const { return m_b; }
};

class Default
{
private:
    int m_a; // note: no default initialization value
    int m_b {};

public:
    Default() = default; // explicitly defaulted default constructor

    int a() const { return m_a; }
    int b() const { return m_b; }
};

class Implicit
{
private:
    int m_a; // note: no default initialization value
    int m_b {};

public:
    // implicit default constructor

    int a() const { return m_a; }
    int b() const { return m_b; }
};

int main()
{
    User user{}; // default initialized
    std::cout << user.a() << ' ' << user.b() << '\n';

    Default def{}; // zero initialized, then default initialized
    std::cout << def.a() << ' ' << def.b() << '\n';
}
```

```

Implicit imp{}; // zero initialized, then default initialized
std::cout << imp.a() << ' ' << imp.b() << '\n';

return 0;
}

```

У автора программа выше выводит:

```

62 0
0 0
0 0

```

Обратите внимание, что `user.a` не был инициализирован нулями до инициализации по умолчанию и, таким образом, остался неинициализированным.

На практике это не должно иметь значения, поскольку вы должны предоставлять инициализаторы по умолчанию для всех элементов данных!

Подсказка

Для класса, в котором нет предоставленного пользователем конструктора по умолчанию, инициализация значением сначала обнуляет класс, а инициализация по умолчанию — нет. Учитывая это, инициализация по умолчанию может быть более производительной, чем инициализация значением (за счёт меньшей безопасности). Если вы хотите повысить производительность в разделе кода, в котором инициализируется множество объектов, в которых нет предоставленных пользователем конструкторов по умолчанию, возможно, стоит изменить эти объекты на инициализированные по умолчанию. В качестве альтернативы можно попробовать изменить класс, добавив в него конструктор по умолчанию с пустым телом. Это позволяет избежать инициализации нулём при использовании инициализации значением, но может препятствовать другим оптимизациям.

- До C++20 класс с пользовательским конструктором по умолчанию (даже если у него есть пустое тело) не является совокупностью, в то время как явно заданный конструктор по умолчанию является совокупностью. Если бы класс в остальном был совокупностью, то первый вариант привёл бы к инициализации списка вместо инициализации совокупности. Начиная с C++20, это несоответствие было устранено, и оба варианта делают класс совокупностью.

Создавайте конструктор по умолчанию только тогда, когда это имеет смысл

Конструктор по умолчанию позволяет создавать объекты неагрегированного типа класса без значений инициализации, заданных пользователем. Таким образом, конструктор по умолчанию должен быть в классе только в том случае, если имеет смысл создавать объекты типа класса, используя все значения по умолчанию.

Например:

```

#include <iostream>

class Fraction
{
private:
    int m_numerator{ 0 };
    int m_denominator{ 1 };

public:
    Fraction() = default;
    Fraction(int numerator, int denominator)
        : m_numerator{ numerator }
        , m_denominator{ denominator }
    {
    }

    void print() const

```

```

    {
        std::cout << "Fraction(" << m_numerator << ", " << m_denominator << ")\n";
    }
};

int main()
{
    Fraction f1 {3, 5};
    f1.print();

    Fraction f2 {}; // will get Fraction 0/1
    f2.print();

    return 0;
}

```

Для класса, представляющего дробь, имеет смысл разрешить пользователю создавать объекты Fraction без инициализаторов (в этом случае пользователь получит дробь 0/1).

Теперь рассмотрим этот класс:

```

#include <iostream>
#include <string>
#include <string_view>

class Employee
{
private:
    std::string m_name{ };
    int m_id{ };

public:
    Employee(std::string_view name, int id)
        : m_name{ name }
        , m_id{ id }
    {
    }

    void print() const
    {
        std::cout << "Employee(" << m_name << ", " << m_id << ")\n";
    }
};

int main()
{
    Employee e1 { "Joe", 1 };
    e1.print();

    Employee e2 {}; // compile error: no matching constructor
    e2.print();

    return 0;
}

```

Для класса, представляющего сотрудника, нет смысла разрешать создание сотрудников без имени. Таким образом, у такого класса не должно быть конструктора по умолчанию, чтобы при попытке пользователя класса создать такой объект возникала ошибка компиляции.

Делегирование конструкторов

По возможности мы стремимся сократить количество избыточного кода (следуя принципу DRY — «Не повторяйся»).

Рассмотрим следующие функции:

```
void A()
{
    // statements that do task A
}

void B()
{
    // statements that do task A
    // statements that do task B
}
```

Обе функции содержат набор операторов, которые выполняют одну и ту же задачу (задача A). В таком случае мы можем провести рефакторинг следующим образом:

```
void A()
{
    // statements that do task A
}

void B()
{
    A();
    // statements that do task B
}
```

Таким образом, мы удалили избыточный код, который существовал в функциях A() и B(). Это упрощает поддержку нашего кода, так как изменения нужно вносить только в одном месте.

Когда класс содержит несколько конструкторов, очень часто код в каждом конструкторе похож, если не идентичен, и содержит множество повторений. Мы также хотели бы по возможности устранить избыточность конструкторов.

Рассмотрим следующий пример:

```
#include <iostream>
#include <string>
#include <string_view>

class Employee
{
private:
    std::string m_name { "???" };
    int m_id { 0 };
    bool m_isManager { false };

public:
    Employee(std::string_view name, int id) // Employees must have a name and an id
        : m_name{ name }, m_id { id }
    {
        std::cout << "Employee " << m_name << " created\n";
    }

    Employee(std::string_view name, int id, bool isManager) // They can optionally be a
manager
```

```

        : m_name{ name }, m_id{ id }, m_isManager { isManager }
    {
        std::cout << "Employee " << m_name << " created\n";
    }
};

int main()
{
    Employee e1{ "James", 7 };
    Employee e2{ "Dave", 42, true };
}

```

Тело каждого конструктора содержит точно такой же оператор print.

Примечание автора

Как правило, не стоит заставлять конструктор что-то выводить (кроме как в целях отладки), так как это означает, что вы не сможете создать объект с помощью этого конструктора в тех случаях, когда вы не хотите что-то выводить. В этом примере мы делаем это, чтобы проиллюстрировать происходящее.

Конструкторам разрешено вызывать другие функции, в том числе другие функции-элементы класса. Таким образом, мы могли бы провести рефакторинг следующим образом:

```

#include <iostream>
#include <string>
#include <string_view>

class Employee
{
private:
    std::string m_name { "???" };
    int m_id{ 0 };
    bool m_isManager { false };

    void printCreated() const // our new helper function
    {
        std::cout << "Employee " << m_name << " created\n";
    }

public:
    Employee(std::string_view name, int id)
        : m_name{ name }, m_id { id }
    {
        printCreated(); // we call it here
    }

    Employee(std::string_view name, int id, bool isManager)
        : m_name{ name }, m_id{ id }, m_isManager { isManager }
    {
        printCreated(); // and here
    }
};

int main()
{
    Employee e1{ "James", 7 };
    Employee e2{ "Dave", 42, true };
}

```

Хотя это лучше, чем предыдущая версия (поскольку избыточный оператор был заменён избыточным вызовом функции), это требует добавления новой функции. И оба наших конструктора также инициализируют `m_name` и `m_id`. В идеале мы бы и эту избыточность убрали.

Можем ли мы сделать лучше? Можем. Но именно здесь многие начинающие программисты сталкиваются с трудностями.

Вызов конструктора в теле функции создает временный объект

Аналогично тому, как в приведённом выше примере мы вызывали функцию B() из функции A(), очевидным решением будет вызвать конструктор Employee(std::string_view, int) из тела Employee(std::string_view, int, bool) для инициализации m_name, m_id, и вывода оператора. Вот как это выглядит:

```
#include <iostream>
#include <string>
#include <string_view>

class Employee
{
private:
    std::string m_name { "???" };
    int m_id { 0 };
    bool m_isManager { false };

public:
    Employee(std::string_view name, int id)
        : m_name{ name }, m_id { id } // this constructor initializes name and id
    {
        std::cout << "Employee " << m_name << " created\n"; // our print statement is
back here
    }

    Employee(std::string_view name, int id, bool isManager)
        : m_isManager { isManager } // this constructor initializes m_isManager
    {
        // Call Employee(std::string_view, int) to initialize m_name and m_id
        Employee(name, id); // this doesn't work as expected!
    }

    const std::string& getName() const { return m_name; }
};

int main()
{
    Employee e2{ "Dave", 42, true };
    std::cout << "e2 has name: " << e2.getName() << "\n"; // print e2.m_name
}
```

Но это работает некорректно, так как программа выводит следующее:

```
Employee Dave created
e2 has name: ???
```

Несмотря на то, что Employee Dave created напечатано, после e2 завершения построения e2.m_name по-прежнему отображается с начальным значением "???". Как такое возможно?

Мы ожидали, что Employee(name, id) вызовет конструктор, чтобы продолжить инициализацию текущего неявного объекта (e2). Но инициализация объекта класса завершается после выполнения списка инициализаторов элементов. К тому времени, когда мы начинаем выполнять тело конструктора, уже слишком поздно выполнять дополнительную инициализацию.

При вызове из тела функции то, что выглядит как вызов функции-конструктора, обычно создаёт и инициализирует временный объект (в другом случае вы получите ошибку компиляции). В приведённом выше примере Employee(name, id); создаёт временный (безымянный) объект Employee. Этот временный объект — тот, для которого m_name установлено значение Dave, и тот, который выводит Employee Dave created. Затем

временный объект уничтожается. e2 Его m_name или m_id никогда не изменяются по сравнению со значениями по умолчанию.

Наилучшая практика

Конструкторы не следует вызывать напрямую из тела другой функции. Это приведёт либо к ошибке компиляции, либо к прямой инициализации временного объекта.

Если вам нужен временный объект, предпочтите инициализацию списком (которая даёт понять, что вы собираетесь создать объект).

Итак, если мы не можем вызвать конструктор из тела другого конструктора, то как нам решить эту проблему?

Делегирование конструкторов

Конструкторы могут делегировать (передавать ответственность за) инициализацию другому конструктору из того же типа класса. Этот процесс иногда называют цепочкой конструкторов, а такие конструкторы — делегационными конструкторами.

Чтобы один конструктор делегировал инициализацию другому конструктору, просто вызовите конструктор в списке инициализации элементов:

```
#include <iostream>
#include <string>
#include <string_view>

class Employee
{
private:
    std::string m_name { "???" };
    int m_id { 0 };

public:
    Employee(std::string_view name)
        : Employee{ name, 0 } // delegate initialization to Employee(std::string_view,
int) constructor
    {
    }

    Employee(std::string_view name, int id)
        : m_name{ name }, m_id { id } // actually initializes the members
    {
        std::cout << "Employee " << m_name << " created\n";
    }

};

int main()
{
    Employee e1{ "James" };
    Employee e2{ "Dave", 42 };
}
```

Когда e1 { "James" } инициализируется, вызывается соответствующий конструктор Employee(std::string_view) с параметром name, равным "James". Список инициализации элементов этого конструктора делегирует инициализацию другому конструктору, поэтому затем вызывается Employee(std::string_view, int). Значение name ("James") передаётся в качестве первого аргумента, а литерал 0 передаётся в качестве второго аргумента. Затем список инициализации элементов делегированного конструктора инициализирует элементы. Затем выполняется тело делегированного конструктора. Затем управление возвращается к исходному конструктору, чьё (пустое) тело выполняется. Наконец, управление возвращается к вызывающей стороне.

Недостатком этого метода является то, что иногда требуется дублировать значения инициализации. При делегировании конструктору `Employee(std::string_view, int)` нам нужно значение инициализации для параметра `int`. Нам пришлось жестко закодировать литерал `0`, так как нет возможности сослаться на инициализатор элемента по умолчанию.

Несколько дополнительных замечаний о делегировании конструкторов. Во-первых, конструктор, делегирующий другому конструктору, не может сам выполнять инициализацию элементов. Таким образом, ваши конструкторы могут делегировать или инициализировать, но не то и другое одновременно.

В качестве отступления...

Обратите внимание, что мы использовали `Employee(std::string_view)` (конструктор с меньшим количеством параметров) в качестве делегата `Employee(std::string_view name, int id)` (конструктора с большим количеством параметров). Обычно конструктор с меньшим количеством параметров делегирует конструктору с большим количеством параметров.

Если бы мы вместо этого выбрали делегирование `Employee(std::string_view name, int id)` в `Employee(std::string_view)` то не смогли бы инициализировать `m_id` с помощью `id`, так как конструктор может только делегировать или инициализировать, но не то и другое одновременно.

Во-вторых, один конструктор может делегировать другому конструктору, который, в свою очередь, делегирует первому конструктору. Это образует бесконечный цикл и приведёт к тому, что в вашей программе закончится место в стеке и произойдёт сбой. Этого можно избежать, если убедиться, что все ваши конструкторы не делегируют друг другу.

Наилучшая практика

Если у вас есть несколько конструкторов, подумайте, можно ли использовать делегирующие конструкторы, чтобы сократить количество повторяющегося кода.

Сокращение конструкторов с использованием аргументов по умолчанию

Значения по умолчанию также иногда используются для сокращения количества конструкторов. Например, задав значение по умолчанию для нашего `id` параметра, мы можем создать один `Employee` конструктор, который требует аргумент `name`, но при необходимости принимает аргумент `id`:

```
#include <iostream>
#include <string>
#include <string_view>

class Employee
{
private:
    std::string m_name{};
    int m_id{ 0 }; // default member initializer

public:
    Employee(std::string_view name, int id = 0) // default argument for id
        : m_name{ name }, m_id{ id }
    {
        std::cout << "Employee " << m_name << " created\n";
    }
};

int main()
{
    Employee e1{ "James" };
    Employee e2{ "Dave", 42 };
}
```

Поскольку значения по умолчанию должны быть привязаны к крайним правым параметрам в вызове функции, при определении классов рекомендуется сначала определять элементы, для которых пользователь должен предоставить значения инициализации (а затем сделать их крайними левыми параметрами конструктора). Элементы, для которых пользователь может предоставить значения по желанию (поскольку значения по умолчанию допустимы), следует определять вторыми (а затем сделать их крайними правыми параметрами конструктора).

Наилучшая практика

Элементы, для которых пользователь должен указать значения инициализации, должны быть определены первыми (и в качестве крайних левых параметров конструктора). Элементы, для которых пользователь может указать значения инициализации (поскольку значения по умолчанию допустимы), должны быть определены вторыми (и в качестве крайних правых параметров конструктора).

Обратите внимание, что этот метод также требует дублирования значения инициализации по умолчанию для `m_id` («0»): один раз в качестве инициализатора элемента по умолчанию и один раз в качестве аргумента по умолчанию.

Головоломка: избыточные конструкторы против избыточных значений по умолчанию

В приведённых выше примерах мы использовали делегирующие конструкторы, а затем аргументы по умолчанию, чтобы уменьшить избыточность конструкторов. Но оба этих метода требовали от нас дублирования значений инициализации для наших элементов в разных местах. К сожалению, в настоящее время нет способа указать, что делегирующий конструктор или аргумент по умолчанию должен использовать значение инициализатора элемента по умолчанию.

Существуют разные мнения о том, что лучше: иметь меньше конструкторов (с дублированием значений инициализации) или больше конструкторов (без дублирования значений инициализации). Мы считаем, что обычно проще иметь меньше конструкторов, даже если это приводит к дублированию значений инициализации.

Для продвинутых читателей

Если у нас есть значение инициализации, которое используется в нескольких местах (например, в качестве инициализатора элемента по умолчанию и аргумента по умолчанию для параметра конструктора), мы можем определить именованную константу и использовать её везде, где требуется значение инициализации. Это позволяет определить значение инициализации в одном месте.

Хотя для этого можно использовать глобальную переменную `constexpr`, лучше использовать элемент `static constexpr` внутри класса:

```
#include <iostream>
#include <string>
#include <string_view>

class Employee
{
private:
    static constexpr int default_id { 0 }; // define a named constant with our
desired initialization value

    std::string m_name {};
    int m_id { default_id }; // we can use it here

public:

    Employee(std::string_view name, int id = default_id) // and we can use it here
        : m_name { name }, m_id { id }
    {
        std::cout << "Employee " << m_name << " created\n";
    }
};
```

```
    }  
};  
  
int main()  
{  
    Employee e1 { "James" };  
    Employee e2 { "Dave", 42 };  
}
```

Использование `static` ключевого слова в этом контексте позволяет нам иметь единственный `default_id` элемент, который является общим для всех `Employee` объектов. Без `static` каждый `Employee` объект имел бы свой собственный независимый `default_id` элемент (который работал бы, но был бы пустой тратой памяти). Недостатком такого подхода является то, что каждая дополнительная именованная константа добавляет ещё одно имя, которое нужно понимать, что делает ваш класс немного более запутанным и сложным. Стоит ли это делать, зависит от того, сколько таких констант требуется и в скольких местах нужны значения инициализации.

Мы подробнее рассмотрим статические элементы данных в следующих уроках

Временные объекты класса

Рассмотрим следующий пример:

```
#include <iostream>

int add(int x, int y)
{
    int sum{ x + y }; // stores x + y in a variable
    return sum;       // returns value of that variable
}

int main()
{
    std::cout << add(5, 3) << '\n';

    return 0;
}
```

В функции `add()` переменная `sum` используется для хранения результата выражения `x + y`. Затем эта переменная вычисляется в операторе `return`, чтобы получить возвращаемое значение. Хотя это может быть полезно для отладки (чтобы при необходимости можно было проверить значение `sum`), на самом деле это усложняет функцию, поскольку определяет объект, который затем используется только один раз.

В большинстве случаев, когда переменная используется только один раз, она нам на самом деле не нужна. Вместо этого мы можем подставить выражение, используемое для инициализации переменной, туда, где она использовалась бы. Вот функция `add()` в таком виде:

```
#include <iostream>

int add(int x, int y)
{
    return x + y; // just return x + y directly
}

int main()
{
    std::cout << add(5, 3) << '\n';

    return 0;
}
```

Это работает не только с возвращаемыми значениями, но и с большинством аргументов функций. Например, вместо этого:

```
#include <iostream>

void printValue(int value)
{
    std::cout << value;
}

int main()
{
    int sum{ 5 + 3 };
    printValue(sum);

    return 0;
}
```

Мы можем написать это:

```
#include <iostream>
```

```

void printValue(int value)
{
    std::cout << value;
}

int main()
{
    printValue(5 + 3);

    return 0;
}

```

Обратите внимание, насколько чище становится наш код. Нам не нужно определять переменную и присваивать ей имя. И нам не нужно просматривать всю функцию, чтобы определить, используется ли эта переменная где-то ещё. Поскольку `5 + 3` — это выражение, мы знаем, что оно используется только в этой строке.

Обратите внимание, что это работает только в тех случаях, когда допустимо выражение `rvalue`. В случаях, когда требуется выражение `lvalue`, у нас должен быть объект:

```

#include <iostream>

void addOne(int& value) // pass by non-const references requires lvalue
{
    ++value;
}

int main()
{
    int sum { 5 + 3 };
    addOne(sum); // okay, sum is an lvalue

    addOne(5 + 3); // compile error: not an lvalue

    return 0;
}

```

Объекты временного класса

Та же проблема применима и в контексте типов классов.

Примечание автора

Здесь мы будем использовать класс, но всё, что в этом уроке связано с инициализацией списков, в равной степени применимо к структурам, которые инициализируются с помощью агрегатной инициализации.

Следующий пример аналогичен приведенным выше, но использует программный тип класса `IntPair` вместо `int`:

```

#include <iostream>

class IntPair
{
private:
    int m_x{};
    int m_y{};

public:
    IntPair(int x, int y)
        : m_x { x }, m_y { y }
    {}
}

```

```

    int x() const { return m_x; }
    int y() const { return m_y; }
};

void print(IntPair p)
{
    std::cout << "(" << p.x() << ", " << p.y() << ")\n";
}

int main()
{
    // Case 1: Pass variable
    IntPair p { 3, 4 };
    print(p); // prints (3, 4)

    return 0;
}

```

В случае 1 мы создаём экземпляр переменной `IntPair p` и затем передаём `p` в функцию `print()`.

Однако `p` используется только один раз, а функция `print()` принимает `r`-значения, так что нет необходимости определять здесь переменную. Поэтому давайте избавимся от `p`.

Мы можем сделать это, передав временный объект вместо именованной переменной. Временный объект (иногда называемый анонимным объектом или безымянным объектом) — это объект, у которого нет имени и который существует только в течение выполнения одного выражения.

Существует два распространенных способа создания временных объектов типа класса:

```

#include <iostream>

class IntPair
{
private:
    int m_x{};
    int m_y{};

public:
    IntPair(int x, int y)
        : m_x { x }, m_y { y }
    {}

    int x() const { return m_x; }
    int y() const { return m_y; }
};

void print(IntPair p)
{
    std::cout << "(" << p.x() << ", " << p.y() << ")\n";
}

int main()
{
    // Case 1: Pass variable
    IntPair p { 3, 4 };
    print(p);

    // Case 2: Construct temporary IntPair and pass to function
    print(IntPair { 5, 6 } );

    // Case 3: Implicitly convert { 7, 8 } to a temporary Intpair and pass to function
    print( { 7, 8 } );
}

```

```
    return 0;
}
```

В случае 2 мы указываем компилятору создать объект `IntPair` и инициализировать его с помощью `{ 5, 6 }`. Поскольку у этого объекта нет имени, он является временным. Затем временный объект передаётся в параметр `p` функции `print()`. Когда вызов функции завершается, временный объект уничтожается.

В случае 3 мы также создаём временный объект `IntPair` для передачи в функцию `print()`. Однако, поскольку мы не указали явно, какой тип нужно создать, компилятор выведет необходимый тип (`IntPair`) из параметра функции, а затем неявно преобразует `{ 7, 8 }` в объект `IntPair`.

Подводя итог:

```
IntPair p { 1, 2 }; // create named object p initialized with { 1, 2 }
IntPair { 1, 2 };   // create temporary object initialized with { 1, 2 }
{ 1, 2 };           // compiler will try to convert { 1, 2 } to temporary object
matching expected type (typically a parameter or return type)
```

Еще несколько примеров:

```
std::string { "Hello" }; // create a temporary std::string initialized with "Hello"
std::string {};           // create a temporary std::string using value initialization /
default constructor
```

Временные объекты и возвращаемые значения !!!!

Когда функция возвращает значение, возвращаемый объект является временным (инициализированным с помощью значения или объекта, указанного в операторе `return`).

Вот несколько примеров:

```
#include <iostream>

class IntPair
{
private:
    int m_x{};
    int m_y{};

public:
    IntPair(int x, int y)
        : m_x { x }, m_y { y }
    {}

    int x() const { return m_x; }
    int y() const { return m_y; }
};

void print(IntPair p)
{
    std::cout << "(" << p.x() << ", " << p.y() << ")\n";
}

// Case 1: Create named variable and return
IntPair ret1()
{
    IntPair p { 3, 4 };
    return p; // returns temporary object (initialized using p)
}

// Case 2: Create temporary IntPair and return
IntPair ret2()
```

```

{
    return IntPair { 5, 6 }; // returns temporary object (initialized using another
temporary object)
}

// Case 3: implicitly convert { 7, 8 } to IntPair and return
IntPair ret3()
{
    return { 7, 8 }; // returns temporary object (initialized using another temporary
object)
}

int main()
{
    print(ret1());
    print(ret2());
    print(ret3());

    return 0;
}

```

Случаи в этом примере аналогичны случаям в предыдущем примере.

static_cast - явное создание экземпляра временного объекта

В случаях, когда нам нужно преобразовать значение из одного типа в другой, но при этом не требуется сужающее преобразование, мы часто можем использовать static_cast или явное создание временного объекта.

Например:

```

#include <iostream>

int main()
{
    char c { 'a' };

    std::cout << static_cast<int>( c ) << '\n'; // static_cast returns a temporary int
direct-initialized with value of c
    std::cout << int { c } << '\n';             // explicitly creates a temporary int
list-initialized with value c

    return 0;
}

```

static_cast<int>(c) возвращает временный объект int, который инициализируется напрямую значением c. int { c } создаёт временный объект int, который инициализируется списком со значением c. В любом случае мы получаем временный объект int, инициализированный значением c, что нам и нужно.

Давайте покажем немного более сложный пример:

Строка print.h:

```

#include <iostream>
#include <string>
void printString(const std::string &s)
{
    std::cout << s << '\n';
}

```

.cpp:

```

#include "printString.h"
#include <iostream>

```

```

#include <string>
#include <string_view>

int main()
{
    std::string_view sv { "Hello" };

    // We want to print sv using the printString() function

    // printString(sv); // compile error: a std::string_view won't implicitly convert to
    // a std::string

    printString( static_cast<std::string>(sv) ); // Case 1: static_cast returns a
    temporary std::string direct-initialized with sv
    printString( std::string { sv } );           // Case 2: explicitly creates a
    temporary std::string list-initialized with sv
    printString( std::string ( sv ) );           // Case 3: C-style cast returns
    temporary std::string direct-initialized with sv (avoid this one!)

    return 0;
}

```

Допустим, код в заголовочном файле `printString.h` — это не тот код, который мы можем изменить (например, потому что он распространяется вместе с какой-то сторонней библиотекой, которую мы используем, и был написан для совместимости с C++11, который не поддерживает `std::string_view`). Как же тогда вызвать `printString()` с помощью `sv`? Поскольку `std::string_view` не преобразуется неявно в `std::string` (из соображений эффективности), мы не можем просто использовать `sv` в качестве аргумента. Мы должны использовать явное преобразование.

В случае 1 `static_cast<std::string>(sv)` вызывает оператор `static_cast` для приведения `sv` к `std::string`. Это возвращает временное `std::string` значение, которое было инициализировано напрямую с помощью `sv`, а затем используется в качестве аргумента для вызова функции.

В случае 2 `std::string { sv }` создаёт временный `std::string` объект, который инициализируется списком с помощью `sv`. Поскольку это явная конструкция, преобразование разрешено. Затем этот временный объект используется в качестве аргумента для вызова функции.

В случае 3 `std::string ( sv )` используйте приведение типов в стиле C, чтобы преобразовать `sv` в `std::string`. Хотя здесь это работает, приведение типов в стиле C в целом может быть опасным, и его следует избегать.

Наилучшая практика

В качестве краткого эмпирического правила: предпочитайте `static_cast` при преобразовании в базовый тип и временный, инициализированный списком, при преобразовании в тип класса. Используйте `static_cast` для создания временного объекта, если выполняется любое из следующих условий:

Нам нужно выполнить сужающее преобразование.

Мы хотим, чтобы было очевидно, что мы преобразуем тип, что приведёт к некоторому другому поведению (например, `char` в `int`).

По какой-то причине мы хотим использовать прямую инициализацию (например, чтобы избежать приоритета конструкторов списков).

При выполнении любого из следующих условий лучше создать новый объект (используя инициализацию списка) для создания временного объекта:

Мы хотим использовать инициализацию списка (например, для защиты от сужающих преобразований или потому, что нам нужно вызвать конструктор списка).

Нам нужно предоставить конструктору дополнительные аргументы, чтобы облегчить преобразование.

Введение в конструктор копирования

Рассмотрим следующую программу:

```
#include <iostream>

class Fraction
{
private:
    int m_numerator{ 0 };
    int m_denominator{ 1 };

public:
    // Default constructor
    Fraction(int numerator=0, int denominator=1)
        : m_numerator{numerator}, m_denominator{denominator}
    {
    }

    void print() const
    {
        std::cout << "Fraction(" << m_numerator << ", " << m_denominator << ")\n";
    }
};

int main()
{
    Fraction f { 5, 3 }; // Calls Fraction(int, int) constructor
    Fraction fCopy { f }; // What constructor is used here?

    f.print();
    fCopy.print();

    return 0;
}
```

Возможно, вы удивитесь, обнаружив, что эта программа прекрасно компилируется и выдаёт результат:

```
Fraction(5, 3)
Fraction(5, 3)
```

Давайте подробнее рассмотрим, как работает эта программа.

Инициализация переменной `f` — это просто стандартная инициализация фигурными скобками, которая вызывает конструктор `Fraction(int, int)`

Но что насчёт следующей строки? Инициализация переменной `fCopy` — это тоже инициализация, и вы знаете, что функции-конструкторы используются для инициализации классов. Так какой же конструктор вызывает эта строка?

Ответ таков: конструктор копирования.

Конструктор копирования

Конструктор копирования — это конструктор, который используется для инициализации объекта существующим объектом того же типа. После выполнения конструктора копирования вновь созданный объект должен быть копией объекта, переданного в качестве инициализатора.

Неявный конструктор копирования

Если вы не предоставите конструктор копирования для своих классов, C++ создаст для вас общедоступный неявный конструктор копирования. В приведенном выше примере оператор `Fraction fCopy { f };` вызывает неявный конструктор копирования для инициализации `fCopy` с помощью `f`.

По умолчанию неявный конструктор копирования выполняет поэлементную инициализацию. Это означает, что каждый элемент будет инициализирован с помощью соответствующего элемента класса, переданного в качестве инициализатора. В приведенном выше примере **fCopy.m_numerator** инициализируется с помощью **f.m_numerator** (который имеет значение 5), а **fCopy.m_denominator** инициализируется с помощью **f.m_denominator** (который имеет значение 3).

После выполнения конструктора копирования элементы **f** и **fCopy** имеют одинаковые значения, поэтому **fCopy** является копией **f**. Таким образом, вызов **print()** для любого из них даёт одинаковый результат.

Определение вашего собственного конструктора копирования

Мы также можем явно определить собственный конструктор копирования. В этом уроке мы заставим конструктор копирования выводить сообщение, чтобы показать вам, что он действительно выполняется при создании копий.

Конструктор копирования выглядит именно так, как вы и ожидали от него:

```
#include <iostream>

class Fraction
{
private:
    int m_numerator{ 0 };
    int m_denominator{ 1 };

public:
    // Default constructor
    Fraction(int numerator=0, int denominator=1)
        : m_numerator{numerator}, m_denominator{denominator}
    {
    }

    // Copy constructor
    Fraction(const Fraction& fraction)
        // Initialize our members using the corresponding member of the parameter
        : m_numerator{ fraction.m_numerator }
        , m_denominator{ fraction.m_denominator }
    {
        std::cout << "Copy constructor called\n"; // just to prove it works
    }

    void print() const
    {
        std::cout << "Fraction(" << m_numerator << ", " << m_denominator << ")\n";
    }
};

int main()
{
    Fraction f { 5, 3 }; // Calls Fraction(int, int) constructor
    Fraction fCopy { f }; // Calls Fraction(const Fraction&) copy constructor

    f.print();
    fCopy.print();

    return 0;
}
```

Когда эта программа запущена, вы получаете:

```
Fraction(5, 3)
```

Конструктор копирования, который мы определили выше, функционально эквивалентен тому, который мы получили бы по умолчанию, за исключением того, что мы добавили оператор вывода, чтобы доказать, что конструктор копирования действительно вызывается. Этот конструктор копирования вызывается, когда `fCopy` инициализируется с помощью `f`.

Напоминание

Средства контроля доступа работают на уровне класса (а не на уровне объекта). Это означает, что функции-элементы класса могут получать доступ к закрытым элементам любого объекта класса того же типа (а не только к неявному объекту).

Мы используем это в своих интересах в приведённом выше конструкторе копирования `Fraction` для прямого доступа к закрытым элементам `fraction` параметра. В противном случае у нас не было бы возможности напрямую обращаться к этим элементам (без добавления функций доступа, что мы могли бы не захотеть делать).

Конструктор копирования не должен делать ничего, кроме копирования объекта. Это связано с тем, что в некоторых случаях компилятор может оптимизировать конструктор копирования. Если вы полагаетесь на конструктор копирования для выполнения каких-либо действий, кроме простого копирования, эти действия могут выполняться, а могут и не выполняться.

Наилучшая практика

Конструкторы копирования не должны иметь побочных эффектов, помимо копирования.

Предпочитаете конструктор неявного копирования

В отличие от неявного конструктора по умолчанию, который ничего не делает (и, следовательно, редко является тем, что нам нужно), инициализация по элементам, выполняемая неявным конструктором копирования, обычно является именно тем, что нам нужно. Поэтому в большинстве случаев использование неявного конструктора копирования вполне допустимо.

Наилучшая практика

Предпочитайте неявный конструктор копирования, если только у вас нет конкретной причины создавать свой собственный.

Параметр конструктора копирования должен быть ссылкой

Требуется, чтобы параметр конструктора копирования был ссылкой на **lvalue** или константной ссылкой на **lvalue**. Поскольку конструктор копирования не должен изменять параметр, предпочтительнее использовать константную ссылку на **lvalue**.

Наилучшая практика

Если вы пишете собственный конструктор копирования, параметр должен быть ссылкой на константное значение **lvalue**.

Передача по значению и конструктору копирования

При передаче объекта по значению аргумент копируется в параметр. Если аргумент и параметр относятся к одному типу класса, копия создаётся путём неявного вызова конструктора копирования.

Это проиллюстрировано в следующем примере:

```
#include <iostream>

class Fraction
{
private:
    int m_numerator{ 0 };
    int m_denominator{ 1 };
```

```

public:
    // Default constructor
    Fraction(int numerator = 0, int denominator = 1)
        : m_numerator{ numerator }, m_denominator{ denominator }
    {
    }

    // Copy constructor
    Fraction(const Fraction& fraction)
        : m_numerator{ fraction.m_numerator }
        , m_denominator{ fraction.m_denominator }
    {
        std::cout << "Copy constructor called\n";
    }

    void print() const
    {
        std::cout << "Fraction(" << m_numerator << ", " << m_denominator << ")\n";
    }
};

void printFraction(Fraction f) // f is pass by value
{
    f.print();
}

int main()
{
    Fraction f{ 5, 3 };

    printFraction(f); // f is copied into the function parameter using copy constructor

    return 0;
}

```

На компьютере автора этот пример печатает:

```

Copy constructor called
Fraction(5, 3)

```

В приведённом выше примере вызов `printFraction(f)` передаёт `f` по значению. Конструктор копирования вызывается для копирования `f` из `main` в `f` параметр функции `printFraction()`.

Возврат по значению и конструктор копирования

При возврате по значению создаётся временный объект (содержащий копию возвращаемого значения), который передаётся вызывающей стороне. Если тип возвращаемого значения совпадает с типом возвращаемого объекта, временный объект инициализируется путём неявного вызова конструктора копирования.

Например:

```

#include <iostream>

class Fraction
{
private:
    int m_numerator{ 0 };
    int m_denominator{ 1 };

public:
    // Default constructor
    Fraction(int numerator = 0, int denominator = 1)

```

```

        : m_numerator{ numerator }, m_denominator{ denominator }
    {
    }

    // Copy constructor
    Fraction(const Fraction& fraction)
        : m_numerator{ fraction.m_numerator }
        , m_denominator{ fraction.m_denominator }
    {
        std::cout << "Copy constructor called\n";
    }

    void print() const
    {
        std::cout << "Fraction(" << m_numerator << ", " << m_denominator << ")\n";
    }
};

void printFraction(Fraction f) // f is pass by value
{
    f.print();
}

Fraction generateFraction(int n, int d)
{
    Fraction f{ n, d };
    return f;
}

int main()
{
    Fraction f2 { generateFraction(1, 2) }; // Fraction is returned using copy
    constructor

    printFraction(f2); // f2 is copied into the function parameter using copy
    constructor

    return 0;
}

```

Когда generateFraction возвращает Fraction обратно в main, создаётся временный объект Fraction и инициализируется с помощью конструктора копирования.

Поскольку этот временный объект используется для инициализации Fraction f2, он снова вызывает конструктор копирования, чтобы скопировать временный объект в f2.

И когда f2 передаётся в printFraction(), конструктор копирования вызывается в третий раз.

Таким образом, на машине автора этот пример печатает:

```

Copy constructor called
Copy constructor called
Copy constructor called
Fraction(1, 2)

```

Если вы скомпилируете и запустите приведённый выше пример, то обнаружите, что происходит только один вызов конструктора копирования. Это оптимизация компилятора, известная как исключение копирования.

```

Copy constructor called
Fraction(1, 2)

```

Использование = default для создания конструктора копирования по умолчанию

Если у класса нет конструктора копирования, компилятор неявно создаст его для нас. При желании мы можем явно попросить компилятор создать для нас конструктор копирования по умолчанию, используя синтаксис = default:

```
#include <iostream>

class Fraction
{
private:
    int m_numerator{ 0 };
    int m_denominator{ 1 };

public:
    // Default constructor
    Fraction(int numerator=0, int denominator=1)
        : m_numerator{numerator}, m_denominator{denominator}
    {
    }

    // Explicitly request default copy constructor
    Fraction(const Fraction& fraction) = default;

    void print() const
    {
        std::cout << "Fraction(" << m_numerator << ", " << m_denominator << ")\n";
    }
};

int main()
{
    Fraction f { 5, 3 };
    Fraction fCopy { f };

    f.print();
    fCopy.print();

    return 0;
}
```

Использование = delete для предотвращения копирования

Иногда мы сталкиваемся со случаями, когда не хотим, чтобы объекты определённого класса можно было копировать. Мы можем предотвратить это, пометив функцию конструктора копирования как удалённую с помощью синтаксиса = delete:

```
#include <iostream>

class Fraction
{
private:
    int m_numerator{ 0 };
    int m_denominator{ 1 };

public:
    // Default constructor
    Fraction(int numerator=0, int denominator=1)
        : m_numerator{numerator}, m_denominator{denominator}
    {
    }
}
```

```

// Delete the copy constructor so no copies can be made
Fraction(const Fraction& fraction) = delete;

void print() const
{
    std::cout << "Fraction(" << m_numerator << ", " << m_denominator << ")\n";
}
};

int main()
{
    Fraction f { 5, 3 };
    Fraction fCopy { f }; // compile error: copy constructor has been deleted

    return 0;
}

```

В этом примере, когда компилятор попытается найти конструктор для инициализации fCopy с помощью f, он увидит, что конструктор копирования был удалён. Это приведёт к ошибке компиляции.

В качестве отступления...

Вы также можете запретить публичное создание копий объектов класса, сделав конструктор копирования закрытым (поскольку закрытые функции не могут использоваться публично). Однако закрытый конструктор копирования все равно может использоваться другими участниками класса, поэтому это решение не рекомендуется, если только это не требуется.

Для продвинутых читателей

Правило трёх — это хорошо известный принцип C++, согласно которому, если классу требуется пользовательский конструктор копирования, деструктор или оператор присваивания копирования, то, скорее всего, ему требуются все три. В C++11 это правило было расширено до правила пяти, которое добавляет в список конструктор перемещения и оператор присваивания перемещения.

Несоблюдение правила трёх/правила пяти, скорее всего, приведёт к сбоям в работе кода. Мы вернёмся к правилу трёх и правилу пяти, когда будем рассматривать динамическое выделение памяти.

Конструкторы преобразования и ключевое слово `explicit`

Компилятор неявно преобразует значение одного типа в значение другого типа, если такое преобразование возможно.

Рассмотрим следующий пример:

```
#include <iostream>

class Foo
{
private:
    int m_x{};
public:
    Foo(int x)
        : m_x{ x }
    {
    }

    int getX() const { return m_x; }
};

void printFoo(Foo f) // has a Foo parameter
{
    std::cout << f.getX();
}

int main()
{
    printFoo(5); // we're supplying an int argument

    return 0;
}
```

В этой версии `printFoo` имеет параметр `Foo`, но мы передаём аргумент типа `int`. Поскольку эти типы не совпадают, компилятор попытается неявно преобразовать значение `int 5` в объект `Foo` для вызова функции.

Один из наших типов является определяемым программой типом. В стандарте C++ нет конкретных правил, которые указывали бы компилятору, как преобразовывать значения в (или из) определяемый программой тип.

Вместо этого компилятор проверит, не определили ли мы функцию, которую он может использовать для такого преобразования. Такая функция называется определённым пользователем преобразованием.

Преобразующие конструкторы

В приведённом выше примере компилятор найдёт функцию, которая позволяет преобразовать значение `int 5` в объект `Foo`. Эта функция — конструктор `Foo(int)`.

До этого момента мы обычно использовали конструкторы для явного создания объектов:

```
Foo x { 5 }; // Explicitly convert int value 5 to a Foo
```

Подумайте, что это значит: мы передаём значение `int (5)` и получаем объект `Foo` в ответ.

В контексте вызова функции мы пытаемся решить ту же проблему:

```
printFoo(5); // Implicitly convert int value 5 into a Foo
```

Мы предоставляем `int` значение (5), и нам нужен `Foo` объект взамен. `Foo(int)` Конструктор был разработан именно для этого!

Таким образом, в этом случае при вызове `printFoo(5)` параметр `f` инициализируется копированием с помощью конструктора `Foo(int)` с 5 в качестве аргумента!

Конструктор, который можно использовать для выполнения неявного преобразования, называется преобразующим конструктором. По умолчанию все конструкторы являются преобразующими.

Может быть применено только одно пользовательское преобразование

Теперь рассмотрим следующий пример:

```
#include <iostream>
#include <string>
#include <string_view>

class Employee
{
private:
    std::string m_name{};

public:
    Employee(std::string_view name)
        : m_name{ name }
    {
    }

    const std::string& getName() const { return m_name; }
};

void printEmployee(Employee e) // has an Employee parameter
{
    std::cout << e.getName();
}

int main()
{
    printEmployee("Joe"); // we're supplying an string literal argument

    return 0;
}
```

В этой версии мы заменили наш класс `Foo` на класс `Employee`. `printEmployee` имеет параметр `Employee`, и мы передаём строковый литерал в стиле C. И у нас есть конструктор преобразования: `Employee(std::string_view)`.

Возможно, вы удивитесь, обнаружив, что эта версия не компилируется. Причина проста: для выполнения неявного преобразования может быть применено только одно пользовательское преобразование, а в этом примере их два. Сначала наш строковый литерал в стиле C должен быть преобразован в `std::string_view` (с помощью конструктора преобразования `std::string_view`), а затем наш `std::string_view` должен быть преобразован в `Employee` (с помощью конструктора преобразования `Employee(std::string_view)`).

Есть два способа заставить этот пример работать:

1. Используйте `std::string_view` литерал:

```
int main()
{
    using namespace std::literals;
    printEmployee( "Joe"sv ); // now a std::string_view literal

    return 0;
}
```

Это работает, потому что теперь требуется только одно пользовательское преобразование (из `std::string_view` в `Employee`).

2. Явно сконструируйте Employee, а не неявно создавайте его:

```
int main()
{
    printEmployee(Employee{ "Joe" });

    return 0;
}
```

Это также работает, потому что теперь требуется только одно пользовательское преобразование (из строкового литерала в `std::string_view` для инициализации объекта `Employee`). Передача явно созданного объекта `Employee` в функцию не требует второго преобразования.

В этом последнем примере используется полезная техника: неявное преобразование легко превратить в явное определение. Мы рассмотрим больше примеров этого позже в этом уроке.

Когда преобразование конструкторов идет не так, как надо

Рассмотрим следующую программу:

```
#include <iostream>

class Dollars
{
private:
    int m_dollars{};

public:
    Dollars(int d)
        : m_dollars{ d }
    {
    }

    int getDollars() const { return m_dollars; }
};

void print(Dollars d)
{
    std::cout << "$" << d.getDollars();
}

int main()
{
    print(5);

    return 0;
}
```

Когда мы вызываем `print(5)`, конструктор преобразования `Dollars(int)` будет использоваться для преобразования 5 в объект `Dollars`. Таким образом, эта программа выводит:

```
$5
```

Хотя, возможно, это и было намерением вызывающей стороны, трудно сказать наверняка, потому что вызывающая сторона никак не указала, что именно этого она хотела. Вполне возможно, что вызывающая сторона предполагала, что будет выведено 5, и не ожидала, что компилятор молча и неявно преобразует наше `int` значение в `Dollars` объект, чтобы удовлетворить этот вызов функции.

Хотя этот пример тривиален, в более крупной и сложной программе довольно легко удивиться тому, что компилятор выполняет неявное преобразование, которого вы не ожидали, что приводит к неожиданному поведению во время выполнения.

Было бы лучше, если бы наша функция `print(Dollars)` могла вызываться только с объектом `Dollars`, а не с любым значением, которое может быть неявно преобразовано в `Dollars` (особенно в фундаментальный тип, такой как `int`). Это снизило бы вероятность случайных ошибок.

Ключевое слово `explicit`

Чтобы решить такие проблемы, мы можем использовать ключевое слово `explicit`, чтобы сообщить компилятору, что конструктор не должен использоваться в качестве конструктора преобразования.

Создание конструктора `explicit` имеет два заметных последствия:

- Явный конструктор не может быть использован для выполнения инициализации копирования или инициализации списка копирования.
- Явный конструктор нельзя использовать для неявных преобразований (поскольку для этого используется инициализация копированием или инициализация списком копирования).

Давайте изменим конструктор `Dollars(int)` из предыдущего примера, чтобы он был явным конструктором:

```
#include <iostream>

class Dollars
{
private:
    int m_dollars{};

public:
    explicit Dollars(int d) // now explicit
        : m_dollars{ d }
    {
    }

    int getDollars() const { return m_dollars; }
};

void print(Dollars d)
{
    std::cout << "$" << d.getDollars();
}

int main()
{
    print(5); // compilation error because Dollars(int) is explicit

    return 0;
}
```

Поскольку компилятор больше не может использовать `Dollars(int)` в качестве конструктора преобразования, он не может найти способ преобразовать `5` в `Dollars`. Следовательно, он выдаст ошибку компиляции.

Для конструкторов с отдельным объявлением (внутри класса) и определением (вне класса) ключевое слово `explicit` используется только в объявлении.

Явные конструкторы могут использоваться для прямой инициализации списка

Явный конструктор все еще можно использовать для прямой инициализации списка:

```
// Assume Dollars(int) is explicit
int main()
{
    Dollars d1(5); // ok
    Dollars d2{5}; // ok
}
```

Теперь давайте вернёмся к предыдущему примеру, в котором мы сделали конструктор `Dollars(int)` явным, и поэтому при компиляции возникла ошибка:

```
print(5); // compilation error because Dollars(int) is explicit
```

Что, если мы на самом деле хотим вызвать `print()` со `int` значением 5, но конструктор является явным? Обходной путь прост: вместо того, чтобы неявно преобразовывать компилятор 5 в `Dollars`, который может быть передан `print()`, мы можем сами явно определить `Dollars` объект:

```
print(Dollars{5}); // ok: explicitly create a Dollars
```

Это разрешено, потому что мы по-прежнему можем использовать явные конструкторы для инициализации объектов. И поскольку теперь мы явно создали `Dollars`, тип аргумента соответствует типу параметра, поэтому преобразование не требуется!

Это не только компилируется и запускается, но и лучше документирует наши намерения, поскольку в нём явно указано, что мы хотели вызвать эту функцию с помощью объекта `Dollars`

Возвращение значений и конструкторы с ключевым словом `explicit`

Если возвращаемое функцией значение не соответствует типу возвращаемого значения функции, произойдёт неявное преобразование. Как и в случае с передачей по значению, для таких преобразований нельзя использовать явные конструкторы.

Следующая программа показывает несколько вариантов возвращаемых значений и их результаты:

```
#include <iostream>

class Foo
{
public:
    explicit Foo() // note: explicit (just for sake of example)
    {
    }

    explicit Foo(int x) // note: explicit
    {
    }
};

Foo getFoo()
{
    // explicit Foo() cases
    return Foo{ }; // ok
    return { }; // error: can't implicitly convert initializer list to Foo

    // explicit Foo(int) cases
    return 5; // error: can't implicitly convert int to Foo
    return Foo{ 5 }; // ok
    return { 5 }; // error: can't implicitly convert initializer list to Foo
}

int main()
{
    return 0;
}
```

Возможно, удивительно, но `return { 5 }` считается конвертацией.

Рекомендации по использованию `explicit`

Наилучшая практика

Сделайте любой конструктор, принимающий один аргумент, `explicit` по умолчанию. Если неявное преобразование между типами семантически эквивалентно и эффективно, вы можете сделать конструктор неявным.

Не создавайте явные конструкторы копирования или перемещения, так как они не выполняют преобразования.

Агрегаты и классы constexpr

Функции constexpr, которые могут вычисляться как во время компиляции, так и во время выполнения. Например:

```
#include <iostream>

constexpr int greater(int x, int y)
{
    return (x > y ? x : y);
}

int main()
{
    std::cout << greater(5, 6) << '\n'; // greater(5, 6) may be evaluated at compile-
time or runtime

    constexpr int g { greater(5, 6) }; // greater(5, 6) must be evaluated at compile-
time
    std::cout << g << '\n';           // prints 6

    return 0;
}
```

В этом примере greater() — это функция constexpr, а greater(5, 6) — это константное выражение, которое может быть вычислено как во время компиляции, так и во время выполнения. Поскольку std::cout << greater(5, 6) вызывает greater(5, 6) в контексте, отличном от constexpr, компилятор может выбрать, вычислять ли greater(5, 6) во время компиляции или во время выполнения. Когда greater(5, 6) используется для инициализации переменной constexpr g, greater(5, 6) вызывается в контексте constexpr и должен быть вычислен во время компиляции.

Теперь рассмотрим следующий аналогичный пример:

```
#include <iostream>

struct Pair
{
    int m_x {};
    int m_y {};

    int greater() const
    {
        return (m_x > m_y ? m_x : m_y);
    }
};

int main()
{
    Pair p { 5, 6 }; // inputs are constexpr values
    std::cout << p.greater() << '\n'; // p.greater() evaluates at runtime

    constexpr int g { p.greater() }; // compile error: greater() not constexpr
    std::cout << g << '\n';

    return 0;
}
```

В этой версии у нас есть совокупная структура с именем Pair, и greater() теперь является функцией-элементом. Однако, поскольку функция-элемент greater() не является constexpr, p.greater() не является константным выражением. Когда std::cout << p.greater() вызывает p.greater() (в контексте, отличном от constexpr), p.greater() будет вычисляться во время выполнения. Однако, когда мы пытаемся использовать p.greater() для

инициализации переменной constexpr g, мы получаем ошибку компиляции, поскольку p.greater() не может быть вычислена во время компиляции.

Поскольку входные данные для p являются значениями constexpr (5 и 6), кажется, что p.greater() можно вычислить во время компиляции. Но как это сделать?

Функции - элементы constexpr

Как и функции, не являющиеся элементами класса, функции-элементы могут быть объявлены constexpr с помощью ключевого слова constexpr. Функции-элементы constexpr могут вычисляться как во время компиляции, так и во время выполнения.

```
#include <iostream>

struct Pair
{
    int m_x {};
    int m_y {};

    constexpr int greater() const // can evaluate at either compile-time or runtime
    {
        return (m_x > m_y ? m_x : m_y);
    }
};

int main()
{
    Pair p { 5, 6 };
    std::cout << p.greater() << '\n'; // okay: p.greater() evaluates at runtime

    constexpr int g { p.greater() }; // compile error: p not constexpr
    std::cout << g << '\n';

    return 0;
}
```

В этом примере мы сделали greater() функцией constexpr, чтобы компилятор мог вычислить её либо во время выполнения, либо во время компиляции.

Когда мы вызываем p.greater() выражение во время выполнения std::cout << p.greater(), оно вычисляется во время выполнения.

Однако, когда p.greater() используется для инициализации переменной constexpr g, мы получаем ошибку компилятора. Хотя greater() теперь является constexpr, p по-прежнему не является constexpr, поэтому p.greater() не является константным выражением.

Агрегаты constexpr

Хорошо, если нам нужно, чтобы `p` было `constexpr`, давайте просто сделаем его `constexpr`:

```
#include <iostream>

struct Pair // Pair is an aggregate
{
    int m_x {};
    int m_y {};

    constexpr int greater() const
    {
        return (m_x > m_y ? m_x : m_y);
    }
};

int main()
{
    constexpr Pair p { 5, 6 };          // now constexpr
    std::cout << p.greater() << '\n'; // p.greater() evaluates at runtime or compile-time

    constexpr int g { p.greater() };    // p.greater() must evaluate at compile-time
    std::cout << g << '\n';

    return 0;
}
```

Поскольку `Pair` является совокупностью, а совокупности неявно поддерживают `constexpr`, мы закончили. Это работает! Поскольку `p` является типом `constexpr`, а `greater()` является функцией-элементом `constexpr`, `p.greater()` является константным выражением и может использоваться там, где допустимы только константные выражения.

Объекты класса constexpr и конструкторы constexpr

Теперь давайте сделаем наш `Pair` неагрегатным:

```
#include <iostream>

class Pair // Pair is no longer an aggregate
{
private:
    int m_x {};
    int m_y {};

public:
    Pair(int x, int y): m_x { x }, m_y { y } {}

    constexpr int greater() const
    {
        return (m_x > m_y ? m_x : m_y);
    }
};

int main()
{
    constexpr Pair p { 5, 6 };          // compile error: p is not a literal type
    std::cout << p.greater() << '\n';

    constexpr int g { p.greater() };
    std::cout << g << '\n';
}
```

```
    return 0;
}
```

Этот пример почти идентичен предыдущему, за исключением того, что `Pair` больше не является совокупностью (из-за наличия закрытых элементов данных и конструктора).

Когда мы компилируем эту программу, мы получаем сообщение об ошибке компилятора о том, что `Pair` не является «буквальным типом». Что вы сказали?

В C++ буквальный тип — это любой тип, для которого можно создать объект в рамках константного выражения. Другими словами, объект не может быть `constexpr`, если его тип не является буквальным. А наш неагрегированный `Pair` тип таковым не является.

Исправить это просто: мы просто создадим конструктор `constexpr`:

```
#include <iostream>

class Pair
{
private:
    int m_x {};
    int m_y {};

public:
    constexpr Pair(int x, int y): m_x { x }, m_y { y } {} // now constexpr

    constexpr int greater() const
    {
        return (m_x > m_y ? m_x : m_y);
    }
};

int main()
{
    constexpr Pair p { 5, 6 };
    std::cout << p.greater() << '\n';

    constexpr int g { p.greater() };
    std::cout << g << '\n';

    return 0;
}
```

Это работает так, как ожидалось, точно так же, как это делала наша агрегированная версия `Pair`.

Наилучшая практика

Если вы хотите, чтобы ваш класс можно было вычислить во время компиляции, сделайте функции-элементы и конструктор `constexpr`.

Неявно определённые конструкторы являются `constexpr`, если их можно определить как `constexpr`. Явно определённые конструкторы по умолчанию должны быть явно определены как `constexpr`.

Подсказка

`Constexpr` является частью интерфейса класса, и его удаление впоследствии нарушит работу тех, кто вызывает функцию в постоянном контексте.